第 12 章：内容生成 Agent

版本: v2.5 (2026-03-23 全书完成) **作者**: 内容撰写专家（场景篇）
状态: review（待技术审核）
最后更新: 2026-03-23

本章涉及面试题

1. **如何保持长文本生成的一致性？**（涉及 12.2 节）
2. **如何控制生成内容的风格？有哪些方法？**（涉及 12.3 节）
3. **如何解析复杂版面的 PDF 文档（多栏、表格、图文混排）？**（涉及 12.6 节）
4. **OCR 识别错误如何纠错？有哪些方法？**（涉及 12.6 节）
5. **生成内容质量如何检查和保证？**（涉及 12.4 节）

本章概述

学习目标： - 理解内容生成 Agent 的核心挑战：长文本一致性、风格控制、结构化输出 - 掌握上下文管理策略和质量检查机制 - 能够设计多模态内容处理流程（PDF 解析、OCR 纠错、向量对齐） - 理解内容生成 Agent 的最佳实践与常见陷阱

核心知识点： - 内容生成场景分类与核心挑战 - 上下文管理（分块生成、RAG 检索、工作记忆、一致性检查） - 风格控制（系统提示词、Few-shot 示例、风格检查） - 质量检查（自动审核、人工审核、质量追溯） - 多模态内容处理（PDF 复杂版面解析、OCR 纠错、多模态向量对齐）

12.1 需求分析

总：内容生成 Agent 分为长文本生成、结构化生成、风格化生成、代码生成四类，核心挑战是保持长文本一致性和控制生成风格，需要主动管理上下文和持续注入风格约束。

12.1.1 内容生成场景分类

内容生成任务根据输出形式和质量要求可分为四类：

| 场景类型 | 典型应用 | 核心挑战 | 技术重点 | |---------|---------|---------|---------| | **长文本生成** | 文章、报告、小说、剧本 | 一致性保持 | 上下文管理、RAG 检索 | | **结构化生成** | JSON、XML、表格、代码 | 格式规范性 | **Output Parser**（输出解析器）、容错处理 | | **风格化生成** | 营销文案、邮件、社交媒体 | 风格一致性 | 系统提示词、**Few-shot**（少量示

例学习） | | **代码生成** | 代码编写、解释、转换 | 准确性、可执行性 | 类型检查、测试验证 |

漫剧案例：漫剧章节正文属于长文本生成，需要保持角色设定（名字/能力/性格）、剧情逻辑（时间线/因果关系）、文风（语言风格/叙事视角）三者一致。

> **关键定义**：长文本一致性指在 5000-100000 字的内容中，角色设定、剧情逻辑、文风风格保持连贯，不出现前后矛盾。

12.1.2 核心挑战：长文本一致性

为什么长文本一致性困难?

LLM（大型语言模型）生成是逐 **token**（文本最小单元，0.75 个英文单词或 0.5 个汉字）的概率预测过程，本身无状态，存在三个固有限制：

1. **上下文窗口限制**：主流模型上下文窗口有限（GPT-4 8K/32K/128K，Claude 200K），但长文本（如 10 万字小说）远超窗口限制，无法一次性生成
2. **注意力衰减**：即使在大窗口内，LLM 对早期内容的注意力会随距离衰减，导致「忘记」早期设定
3. **概率漂移**：逐 token 生成中，微小的概率偏差累积可能导致风格逐渐偏离

一致性三个维度：

```
┌──────────────────────────────────────────────┐ │ 长文本一致性三维度 │
├──────────────────────────────────────────────┤ │ 1. 角色设定一致性 │ │
- 名字、外貌、能力、性格不矛盾 │ │ - 例：角色 A 不能第 3 章火系第 10 章水
系 │ ├──────────────────────────────────────────────┤ │ 2. 剧情逻辑一致
性 │ │ - 时间线、因果关系、地理位置不矛盾 │ │ - 例：不能「同时出现在两地」
│ ├──────────────────────────────────────────────┤ │ 3. 文风一致性 │ │
- 语言风格、叙事视角、情感基调统一 │ │ - 例：不能前文正式后文卖萌 │
└──────────────────────────────────────────────┘
```

解决方案概览： - **RAG**（检索增强生成）检索相关设定注入上下文 - 工作记忆记录已生成内容的关键信息 - 生成后进行一致性检查（规则+LLM）

12.1.3 核心挑战：风格控制

问题：LLM 默认风格中性，如何引导输出特定风格?

为什么需要主动控制风格?

LLM 训练数据来自多样化来源，默认输出风格是「平均化」的。实际应用场景需要特定风格： - 漫剧正文：第三人称有限视角，语言简洁，情感基调轻松 - 技术文档：正式、准确、无歧义 - 营销文案：吸引眼球、情感驱动、行动号召

风格控制三个维度：

| 维度 | 说明 | 示例 | |------|------|------| | **语言风格** | 正式/casual、简洁/详细、幽默/严肃 | 「简洁直接，不用网络流行语」 | | **叙事视角** | 第一人称/第三人称、全知/有限视角 | 「第三人称有限视角，只描述主角所见」 | | **情感基调** | 轻松/严肃、积极/消极、温暖/冷峻 | 「情感基调轻松，适当加入幽默元素」 |

控制方法： 1. **系统提示词设定**：明确描述期望风格 2. **Few-shot 示例**：展示 1-3 个符合风格的示例 3. **风格检查与修正**：生成后用 LLM 检查风格符合度

> **最佳实践**：长文本生成中需要定期重新注入风格描述（如每 2K token），防止风格漂移。

12.2 方案设计：上下文管理

总：长文本生成需要分块生成（每块 1K-4K token），结合 RAG 检索静态设定、工作记忆记录动态内容、一致性检查发现矛盾，四者协同保证长文本一致性。

12.2.1 长文本分块生成

问题：上下文窗口有限，如何生成超长文本?

解决方案：将长文本分块生成，每块独立生成但保持上下文衔接。

分块策略：

| 分块维度 | 说明 | 适用场景 | |---------|------|---------| | **按章节** | 每章独立生成 | 小说、剧本（天然分章） | | **按场景** | 每场景独立生成 | 剧本、影视脚本 | | **按段落** | 每段落独立生成 | 报告、文章 |

块大小选择：

```
块大小选择权衡： ┌──────────────────────────────────────────────────────┐ │ 块过大（>4K token） │ │ → 质量下降（LLM 注意力分散） │ │ → 成本增加（单次调用 Token 多） │ │ → 错误影响大（整块需重新生成） │ ├──────────────────────────────────────────────────────┤ │ 块过小（<1K token） │ │ → 衔接困难（上下文不足） │ │ → 效率低（调用次数多） │ │ → 风格易漂移（每块独立生成） │ ├──────────────────────────────────────────────────────┤ │ 推
```

```
荐块大小：1K-4K token │ │ → 平衡质量、成本、衔接难度 │
└──────────────────────────────────────────────────┘
```

衔接技巧： - **前情回顾**：每块开头用 1-2 句回顾前一块关键内容（「上一段讲到角色 A 发现了神秘洞穴...」） - **预留衔接点**：每块结尾预留悬念或过渡（「正当他准备进入洞穴时，身后传来了脚步声...」） - **上下文传递**：将前一块的关键信息（角色状态、剧情进展）作为下一块的输入

漫剧案例：漫剧每章分 3-5 块生成，每块 2K token。第 10 章第 2 块开头：「上一段讲到角色 A 用火系能力击退了敌人，但自己也受了轻伤。此刻他正躲在山洞中疗伤...」

12.2.2 RAG 检索增强

为什么需要 RAG?

分块生成中，每块只能看到有限的上下文（前一块内容），但可能需要早期设定（如第 1 章定义的角色能力）。RAG 通过检索相关设定注入上下文，解决「遗忘」问题。

检索内容:

| 内容类型 | 存储位置 | 检索时机 | |---------|---------|---------| | **角色设定** | 向量数据库 | 生成涉及该角色的内容前 | | **世界观设定** | 向量数据库 | 生成新场景前 | | **前文剧情** | 向量数据库 + 工作记忆 | 生成新章节前 | | **风格示例** | 向量数据库 | 每块生成前 |

检索策略:

```
生成请求 → 检索什么？ → 向量检索角色设定/世界观/前文 → Top-K=3-5 条 → 注入上下文 → LLM 生成
```

检索量控制： - **Top-K=3-5**：K<3 信息不足，K>5 干扰生成 - **相似度阈值**：只检索相似度 >0.7 的内容，避免无关信息 - **去重**：检索结果去重，避免重复注入

> **注意**：RAG 检索的是静态设定（角色/世界观），动态生成的内容（剧情进展）需要工作记忆记录。

12.2.3 工作记忆记录

工作记忆与 RAG 的区别:

| 维度 | RAG 检索 | 工作记忆 | |------|---------|---------| | **内容类型** | 静态设定（角色/世界观） | 动态内容（剧情进展/状态变化） | | **更新频率** | 低（设定不常变） | 高（每块生成后更新） | | **存储方式** | 向量数据库 | 结构化存储（JSON/Dict） | | **检索方式** | 向量相似度 | 关键词/实体匹配 |

记录内容： - 角色状态变化（「角色 A 第 10 章获得新能力」） - 剧情进展（「第 10 章结尾主角进入洞穴」） - 伏笔记录（「第 5 章埋下神秘人伏笔，未回收」）

更新机制： `每块生成后： 1. 用 LLM 提取关键信息（Prompt：「从以下文本中提取角色状态变化和剧情进展」） 2. 更新工作记忆（JSON 存储） 3. 衰减旧信息（超过 5 章的信息权重降为 50%）`

漫剧案例：第 10 章第 2 块生成后，工作记忆更新： `json { "角色 A": {"状态": "轻伤", "位置": "山洞", "新能力": "火焰护盾"}, "剧情进展": "进入山洞疗伤，听到脚步声", "伏笔": ["神秘人身份", "洞穴深处秘密"] }`

12.2.4 一致性检查

问题：如何发现生成内容中的矛盾？

检查维度：

| 维度 | 检查方法 | 示例 | |------|---------|------| | **角色设定** | 规则检查 + LLM 检查 | 名字拼写、能力是否与前文一致 | | **剧情逻辑** | LLM 检查 | 时间线顺序正确、因果关系成立 | | **文风一致性** | LLM 检查 + 风格评分 | 语言风格是否偏离、叙事视角是否一致 |

检查时机： - **块级检查**：每块生成后 10 秒内检查（快速发现问题） - **章级检查**：整章完成后 5 分钟内全面检查（系统性检查）

检查方法：

规则检查（适用于明确规则）： `1. 名字拼写检查：检索所有角色名字，检查是否有拼写变异 2. 时间线检查：提取所有时间表述，检查是否顺序正确 3. 数字检查：检查数字是否前后一致（如「三人」vs「四人」）`

LLM 检查（适用于复杂逻辑）： ```` Prompt 示例： 「以下是漫剧第 10 章内容和角色设定，请检查： 1. 角色 A 的能力是否与前文一致？ 2. 时间线是否有冲突？ 3. 是否有其他逻辑矛盾？

角色设定：[检索结果] 第 10 章内容：[生成内容]

请列出发现的矛盾（如有）。」 ```

修复机制： - **轻微矛盾**：自动修复（如名字拼写错误） - **中等矛盾**：重新生成矛盾部分 - **严重矛盾**：转人工审核

12.3 方案设计：风格控制

总：风格控制通过系统提示词具体描述风格、Few-shot 示例展示风格、风格检查与修正确保符合度，三者结合防止长文本中的风格漂移。

12.3.1 系统提示词设定

问题：如何用文字描述期望风格?

风格描述五要素：

| 要素 | 说明 | 示例 | |------|------|------| | **语言风格** | 正式程度、简洁度、幽默度 | 「语言简洁直接，不用网络流行语」 | | **叙事视角** | 人称、视角范围 | 「第三人称有限视角，只描述主角所见所感」 | | **情感基调** | 整体情感色彩 | 「情感基调轻松，适当加入幽默元素」 | | **禁止事项** | 明确不允许的内容 | 「不用感叹号堆砌，不直接描述角色心理」 | | **必须事项** | 必须包含的元素 | 「每章结尾留悬念，对话占 40% 以上」 |

漫剧系统提示词示例：```你是一位专业漫剧编剧，请按照以下风格要求生成内容：

【语言风格】- 语言简洁直接，句子长度 15-25 字 - 不用网络流行语，不卖萌 - 动作描写具体，避免抽象形容词

【叙事视角】- 第三人称有限视角（主角视角） - 只描述主角所见所感，不描述其他角色心理

【情感基调】- 轻松为主，适当加入幽默 - 紧张场景用短句加快节奏

【禁止事项】- 不用感叹号堆砌（每段≤1 个） - 不直接描述角色心理（用动作/对话暗示）

【必须事项】- 每章结尾留悬念 - 对话占 40% 以上篇幅```

> **最佳实践**：长文本生成中每 2K token 重新注入系统提示词，防止风格漂移。

12.3.2 Few-shot 示例

为什么需要 Few-shot?

系统提示词描述抽象风格，LLM 可能理解偏差。Few-shot 示例通过具体片段展示期望风格，LLM 可模仿学习。

示例选择标准：- **代表性**：最能代表期望风格的片段 - **多样性**：覆盖不同场景（对话/动作/描写） - **质量**：已通过人工审核的优质片段

示例数量： - **1-3 个**：<1 个效果不佳，>3 个占用上下文 - **长度**：每个示例 200-500 字（展示风格即可）

示例格式： ``` 【示例 1：对话场景】 输入：主角与配角初次见面，互相试探 输出：[已通过审核的对话片段]

【示例 2：动作场景】 输入：主角使用能力战斗 输出：[已通过审核的动作描写片段] ```

示例更新机制： - 定期（如每周）更新示例库 - 加入新的优质片段 - 淘汰过时片段（风格变化后）

12.3.3 风格检查与修正

问题：如何量化评估风格符合度?

风格检查 Prompt 示例： ``` 请评估以下文本是否符合指定风格：

【风格要求】 - 语言简洁直接，句子长度 15-25 字 - 第三人称有限视角 - 情感基调轻松

【待评估文本】 [生成内容]

【评分维度】 1. 语言风格（0-30 分） 2. 叙事视角（0-30 分） 3. 情感基调（0-20 分） 4. 禁止事项（0-20 分，违反扣分）

【输出格式】 总分：XX/100 问题列表：[如有] 修改建议：[如有] ```

修正机制：

| 评分区间 | 处理方式 | |---------|---------| | **≥80 分** | 自动通过 | | **60-79 分** | 重新生成（注入风格要求） | | **<60 分** | 转人工修改 |

漫剧案例：第 10 章生成后风格检查评分 72 分，问题：「对话占比 35%<40% 要求」「感叹号>5 个/段」。重新生成时注入「对话占 40% 以上，感叹号≤1 个/段」要求，二次评分 85 分通过。

12.4 方案设计：质量检查

总：质量检查包括自动审核（规则+LLM 检查 + 评分）、人工审核（触发条件 + 审核界面 + 反馈）、质量追溯（版本记录 + 问题追溯 + 回滚），三层机制确保内容质量稳定。

12.4.1 自动审核

审核维度与权重：

| 维度 | 权重 | 检查方法 | |------|------|---------| | **一致性** | 30 分 | LLM 检查角色/剧情/文风一致性 | | **风格** | 30 分 | LLM 检查风格符合度 | | **格式** | 20 分 | 规则检查（段落/标点/敏感词） | | **敏感内容** | 20 分 | 敏感词库 + LLM 识别 |

评分阈值： - **≥80 分**：自动通过 - **60-79 分**：返工修改（自动重新生成） - **<60 分**：转人工审核

自动审核流程： `生成内容 → 一致性检查 → 风格检查 → 格式检查 → 敏感内容检查 → 综合评分 │ ├── ≥80 → 自动通过 ├── 60-79 → 返工修改 └── <60 → 转人工审核`

12.4.2 人工审核

触发条件： - 自动审核评分<60 分 - 核心章节（第一章、结局、关键转折） - 用户明确要求人工审核

审核界面要素： - 生成内容展示 - 自动审核评分及各维度得分 - 问题标注（高亮显示矛盾/风格问题） - 审核操作按钮（通过/修改后通过/返工/转交）

审核反馈机制： - 记录审核意见（文本） - 标注问题类型（一致性/风格/格式/敏感） - 反馈用于优化自动审核规则

> **最佳实践**：人工审核意见结构化存储，定期分析高频问题，优化自动审核规则。

12.4.3 质量追溯

版本记录内容： - 版本号（v1.0, v1.1, v2.0...） - 生成/修改时间 - 操作人（系统/人工） - 变更内容（diff） - Prompt 版本 - 模型版本

问题追溯： `发现质量问题 ↓ 查询版本记录 ↓ 定位问题批次（如第 10 章 v1.2） ↓ 追溯 Prompt 版本（如 Prompt v2.1） ↓ 追溯模型版本（如 GPT-4-0613） ↓ 分析原因（如 Prompt v2.1 风格描述模糊） ↓ 修复（回滚到 Prompt v2.0 重新生成）`

回滚机制： - 支持回滚到任意历史版本 - 回滚后重新生成需记录原因 - 保留所有版本用于分析

12.5 最佳实践与陷阱

12.5.1 最佳实践

| 实践 | 说明 | |------|------| | **上下文管理完善** | 分块生成+RAG 检索 + 工作记忆+一致性检查四层机制 | | **风格控制具体** | 系统提示词五要素完整+Few-shot 示例 1-3 个 + 风格检查评分 | | **质量检查多层** | 自动审核（评分阈值）+ 人工审核（核心章节）+ 质量追溯（版本管理） | | **文档齐全** | 风格指南、质量检查标准、问题处理手册 |

12.5.2 常见陷阱

| 陷阱 | 后果 | 解决方案 | |------|------|---------| | **一致性丢失** | 长文本中忘记早期设定 | RAG 检索 + 工作记忆+一致性检查 | | **风格漂移** | 生成中风格逐渐偏离 | 每 2K token 重新注入系统提示词 + 风格检查 | | **格式错误** | 结构化输出格式不规范 | Output Parser+ 正则提取+LLM 修复兜底 | | **质量波动** | 不同批次质量差异大 | 固定 Prompt 版本 + 质量追溯分析 | | **JSON 不规范** | LLM 输出 JSON 格式错误 | 正则提取 JSON+LLM 修复兜底（错误率从 30% 降至 5%） |

12.6 多模态内容处理

总：多模态内容处理包括 PDF 复杂版面解析（多栏检测/表格还原/图文混排）、OCR 纠错（语言模型/规则匹配/置信度阈值）、多模态向量对齐（CLIP 模型）、图像描述生成（视觉模型 + 结构化输出）。

12.6.1 PDF 复杂版面解析

问题：PDF 文档常含多栏布局、表格、图文混排，如何正确解析？

多栏检测：

| 方法 | 原理 | 参数建议 | |------|------|---------| | **空白区域分析** | 扫描水平方向空白带，连续空白>50 像素判定为栏间距 | 栏间距阈值 30-100 像素（根据 DPI 调整） | | **垂直线检测** | 识别页面中的垂直分隔线（显式或隐式） | 最小线长 100 像素 | | **文本块聚类** | 基于 x 坐标聚类文本块，形成 2-3 个栏位分组 | 聚类半径 50 像素 |

表格结构还原：
`步骤： 1. 边界识别：检测水平线和垂直线交点，确定表格边界 2. 行列结构：基于线间距推断行数和列数（无线表格用文本对齐推断） 3. 合并单元格：检测跨越多行/列的空白区域，标记为合并单元格 4. 内容提取：按行列顺序提取单元格内容，输出为 Markdown 表格或 HTML`

图文混排处理： - **图片定位**：检测图片边界框（bounding box），记录坐标和尺寸 - **相对位置**：计算图片与周围文本块的距离，判断是「嵌入」「环绕」还是「独立」 - **说明文字识别**：查找图片附近（上下 50 像素内）的短文本块，通常是「图 1」「Figure 1」格式 - **阅读顺序**：按「从上到下、从左到右」原则，结合栏位信息确定阅读顺序

漫剧案例：漫剧剧本解析学术论文 PDF（双栏布局），检测栏间距 60 像素，正确分离左右栏内容，提取 3 个表格和 5 张配图及说明文字。

12.6.2 OCR（光学字符识别）纠错机制

问题：**OCR**（光学字符识别）识别常有错误，如何纠错？

三种纠错方法结合：

| 方法 | 原理 | 适用场景 | 限制 | |------|------|---------|------| | **语言模型纠错** | LLM 检查语义连贯性，修正不通顺句子 | 长文本、有上下文段落 | 专有名词可能被「纠正」为常见词 | | **规则纠错** | 字典匹配、格式验证、上下文一致性检查 | 领域术语、固定格式 | 需要维护词典和规则 | | **置信度阈值** | OCR 引擎返回字符置信度，低于阈值标记待审核 | 所有场景 | 需要人工审核队列 |

语言模型纠错实现： ``` Prompt： 「以下是 OCR 识别的文本，可能有错误，请修正不通顺的地方，保持原意。 注意：专有名词（如人名、地名、术语）不要修改。

OCR 结果：[待纠错文本] 专有名词白名单：[轩辕墨、灵力、筑基、金丹...]

修正后文本：」 ```

规则纠错： - **字典匹配**：建立领域词典（漫剧术语「灵力」「筑基」「元婴」），OCR 结果不在词典中标记可疑 - **格式验证**：日期（YYYY-MM-DD）、数字（千分位）、邮箱用正则验证 - **上下文一致性**：同一实体多次出现，检查拼写是否一致

置信度阈值： - **字符级**：置信度<0.7 标记可疑 - **单词级**：平均置信度<0.8 标记待审核 - **人工审核**：低于阈值的字符高亮显示，提供上下文供人工判断

漫剧案例：漫剧设定文档 OCR 识别「灵力等级：筑基→金丹→元婴」，字典匹配确认术语正确，但「轩辕墨」置信度 0.65 标记待审核，人工确认后保留。

12.6.3 多模态向量对齐

为什么需要对齐？

图像和文本是不同模态，无法直接比较或检索。多模态向量对齐将两者映射到同一向量空间，支持跨模态检索。

CLIP（对比语言 - 图像预训练）： - **对比语言 - 图像预训练**（Contrastive Language-Image Pre-training） - **对齐原理**：训练时配对图像 - 文本，拉近配对向量距离，推远非配对向量 - **输出**：图像和文本都映射到同一向量空间（如 512 维）

应用场景： - 以图搜图（上传封面图，检索相似封面） - 以文搜图（输入剧情描述，检索匹配封面） - 图像描述生成（图像→向量→检索最接近的文本描述）

漫剧案例：漫剧封面生成用 CLIP 将封面图像和剧情描述向量化，检索最匹配的封面方案。

12.6.4 图像描述生成

步骤：
`1. 图像理解：用视觉模型（GPT-4V、Claude Vision）理解图像内容 2. 描述生成：Prompt「详细描述这张图片的内容，包括人物、场景、动作、情感」 3. 结构化输出：将描述结构化为 JSON（人物列表、场景描述、关键元素） 4. 质量检查：检查描述是否遗漏关键元素、是否有幻觉`

结构化输出示例： `json { "人物": [ {"描述": "黑衣男性", "位置": "画面中央", "动作": "行走"} ], "场景": "古代街道，青石板路，两侧木质建筑", "关键元素": ["灯笼", "招牌", "行人"], "情感基调": "宁静，略带神秘" }`

漫剧案例：漫剧参考图片上传后生成结构化描述（主角：黑衣男性，场景：古代街道，动作：行走），用于后续创作参考。

12.7 简单举例

案例设计

- 案例名称：漫剧章节正文生成
- 涉及知识点：内容生成 Agent 的上下文管理、风格控制、质量检查、多模态内容处理
- 案例目标：帮助理解如何根据大纲生成保持角色设定和文风一致的漫剧章节正文
- 案例内容要点：
- 场景描述：根据大纲生成漫剧第 10 章正文，需要保持角色设定一致、文风一致、质量达标
- 技术应用：分 3 块生成每块 2K token 并传递状态，RAG 检索角色设定和前文剧情，系统提示词设定风格并注入 Few-shot 示例，自动审核评分 85 分通过
- 效果说明：生成内容一致性高、风格统一、质量稳定，多模态处理支持参考图片理解
- 注意事项：不展开自动审核的评分细则（见 12.4 节）

知识来源: - LangChain RAG 官方文档 - https://python.langchain.com/docs/use_cases/question_answering - GPT-3 论文：Language Models are Few-Shot Learners (2020) - https://arxiv.org/abs/2005.14165 - CLIP 论文：Learning Transferable Visual Models From Natural Language Supervision (ICML 2021) - https://arxiv.org/abs/2103.00020

修改记录: - v2.2 (2026-03-23): 量化指标 — 模糊表述改为具体数字（字数范围、检查时间、评分阈值、Top-K） - v2.1 (2026-03-23): 术语定义 — 首次出现必定义（LLM、Token、RAG、Output Parser、Few-shot、OCR、CLIP） - v2.0 (2026-03-23): 文字润色 — 句子简化、删除重复、优化结构 - v1.1 (2026-03-22): 根据编辑统筹意见修改 — 规范知识来源格式（2-3 个权威来源） - v1.0 (2026-03-22): 初稿完成